

PERANCANGAN SERTA IMPLEMENTASI PEMBUATAN PROGRAM MANAJEMEN HARGA DAN STOK MENU CAFE BERBASIS COMMAND LINE INTERFACE (CLI) MENGUNAKAN PYTHON

Disusun untuk memenuhi tugas akhir Project-Based Learning (PBL)

Mata Kuliah Algoritma & Struktur Data (TPL2106)



Disusun oleh:

Yasmin Khoirun Nisa'	J0403251071
Muhammad Yusuf Baihaqi Bawono	J0403251096
Asma Mazaya	J0403251105

Program Studi Teknologi Rekayasa Perangkat Lunak

Sekolah Vokasi IPB University

2026

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas limpahan rahmat dan karunia-Nya sehingga kami dapat menyelesaikan laporan akhir Project-Based Learning (PBL) mata kuliah Algoritma & Struktur Data (TPL2106) ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai bentuk pertanggungjawaban atas proyek yang telah kami kerjakan selama satu semester, yaitu pengembangan Sistem Manajemen Menu dan Harga Cafe berbasis Command Line Interface (CLI) menggunakan bahasa pemrograman Python. Sistem ini dirancang untuk membantu pengelolaan data menu café secara efisien dengan menerapkan berbagai konsep struktur data dan algoritma yang telah dipelajari.

Kami mengucapkan terima kasih kepada Ibu Dr. Shelve Nidya Neyman selaku dosen pengampu mata kuliah Algoritma & Struktur Data yang telah memberikan bimbingan dan arahan selama pengerjaan proyek ini. Ucapan terima kasih juga kami sampaikan kepada seluruh anggota kelompok yang telah berkontribusi penuh dalam setiap tahap pengembangan sistem.

Kami menyadari bahwa laporan ini masih jauh dari kesempurnaan. Oleh karena itu, kritik dan saran yang membangun sangat kami harapkan demi perbaikan di masa yang akan datang. Semoga laporan ini dapat bermanfaat bagi semua pihak yang membacanya.

Bogor, Juni 2026

Kelompok 16

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I. PENDAHULUAN.....	4
1.1 Latar Belakang.....	4
1.2 Tujuan Proyek.....	4
1.3 Ruang Lingkup.....	5
BAB II. ANALISIS DAN PERANCANGAN.....	6
2.1 Deskripsi Sistem.....	6
2.2 Struktur Data yang Digunakan.....	6
2.3 Algoritma yang Digunakan.....	7
2.4 Diagram Alir Program (Flowchart).....	8
2.5 Struktur Program.....	9
BAB III. IMPLEMENTASI PROGRAM.....	11
3.1 Tampilan Menu Utama.....	11
3.2 Fitur Create (Tambah Menu).....	11
3.3 Fitur Read (Tampilkan).....	12
3.4 Fitur Update (Ubah Harga dan Stok).....	12
3.5 Fitur Delete (Hapus Menu).....	13
3.6 Fitur Searching.....	13
3.7 Fitur Sorting.....	14
3.8 Penyimpanan Data (File Handling).....	14
3.9 Validasi Input dan Error Handling.....	16
BAB IV. PENGUJIAN DAN ANALISIS.....	18
4.1 Skenario Pengujian.....	18
4.2 Kelebihan Sistem.....	19
4.3 Keterbatasan Sistem.....	20
BAB V. KENDALA DAN SOLUSI.....	21
BAB VI. PEMBAGIAN TUGAS ANGGOTA.....	22
BAB VII. KESIMPULAN DAN SARAN.....	23
Kesimpulan.....	23
Saran Pengembangan.....	23
DAFTAR PUSTAKA.....	24
LAMPIRAN.....	25
Lampiran A. Link GitHub.....	25
Lampiran B. Screenshot Program.....	25
Lampiran C. Struktur Folder Proyek.....	27
Lampiran D. Commit History GitHub.....	29
Lampiran E. Source Code Utama.....	30

BAB I. PENDAHULUAN

1.1 Latar Belakang

Industri kuliner, khususnya bisnis cafe, mengalami pertumbuhan yang pesat di Indonesia. Pengelolaan menu dan harga yang tertata dengan baik menjadi kebutuhan mendasar bagi setiap café agar dapat beroperasi secara efisien dan memberikan pelayanan terbaik kepada pelanggan. Namun, banyak café kecil hingga menengah masih mengandalkan pencatatan manual yang rentan terhadap kesalahan dan kehilangan data.

Permasalahan ini mendorong kami untuk mengembangkan sebuah aplikasi berbasis Command Line Interface (CLI) yang mampu mengelola data menu dan harga cafe secara terstruktur dan persisten. Sistem yang diberi nama MenuCafe Management System ini memungkinkan pemilik kafe untuk dapat menambahkan, mengubah, mencari, mengurutkan, dan menghapus data menu café. Data disimpan secara permanen dalam file teks berformat CSV (Comma-Separated Values) sehingga tetap tersedia ketika program dijalankan kembali.

Program ini sekaligus menjadi sebuah solusi dalam menghadapi perkembangan teknologi terutama dalam membantu perkembangan dunia F&B yang sangat meningkat pesat di Indonesia. Program kami diharapkan dapat membantu terutama bagi UMKM yang sedang mengembangkan bisnis mereka untuk dapat meningkatkan efisiensi serta penjualana mereka.

1.2 Tujuan Proyek

1. Membangun aplikasi manajemen menu café berbasis terminal (CLI) menggunakan bahasa pemrograman Python.
2. Mengimplementasikan struktur data Dictionary untuk penyimpanan dan pencarian data menu secara efisien.
3. Menerapkan konsep file handling agar data menu café tersimpan secara permanen menggunakan file teks berformat TXT/CSV.
4. Mengimplementasikan fitur CRUD (Create, Read, Update, Delete) pada data menu café.

5. Mengimplementasikan fitur searching (pencarian) berbasis key lookup pada struktur Dictionary.
6. Mengimplementasikan fitur sorting (pengurutan) berdasarkan nama, jenis, harga, dan stok menu.
7. Mengembangkan solusi berbasis perangkat lunak yang dapat diterapkan pada permasalahan nyata di lingkungan bisnis kuliner.

1.3 Ruang Lingkup

- Platform: CLI (Console/Terminal)
- Bahasa Pemrograman: Python 3
- Penyimpanan Data: File TXT berformat CSV (MenuCafe.txt)
- Fitur CRUD: Create (tambah menu), Read (tampilkan semua / cari menu), Update (ubah harga, ubah stok), Delete (hapus menu)
- Searching: Sequential Search melalui key lookup pada Dictionary
- Sorting: Sorting multi-kriteria (nama, jenis, harga, stok) menggunakan fungsi `sorted()` dengan lambda comparator
- Validasi Input: Penanganan input tidak valid dengan pesan kesalahan dan permintaan input ulang

BAB II. ANALISIS DAN PERANCANGAN

2.1 Deskripsi Sistem

MenuCafe Management System adalah aplikasi berbasis CLI (Command Line Interface) yang dirancang untuk membantu pengelolaan data menu dan stok café. Sistem ini memungkinkan pengguna—seperti pemilik atau pengelola café—untuk melakukan operasi lengkap terhadap data menu secara interaktif melalui antarmuka teks.

Pada saat program dijalankan, data menu dibaca dari file MenuCafe.txt yang disimpan dalam format CSV. Setiap baris file tersebut merepresentasikan satu item menu dengan empat atribut: nama menu, jenis menu (Makanan/Minuman/Snack), harga, dan stok. Seluruh data kemudian dimuat ke dalam struktur Dictionary di memori program, sehingga operasi pencarian, pengubahan, dan penghapusan dapat dilakukan secara efisien. Setiap perubahan data secara otomatis akan disinkronisasi kembali ke file penyimpanan, menjamin persistensi data.

2.2 Struktur Data yang Digunakan

Struktur Data	Digunakan Untuk	Fungsi
Dictionary (dict)	stock_dict, data	Menyimpan seluruh data menu café. Setiap menu disimpan berdasarkan nama menu (huruf kecil) sebagai key.
Nested Dictionary (Dictionary di dalam Dictionary)	stock_dict[key]	Menyimpan atribut setiap menu, yaitu Nama, Jenis, Harga, dan Stok.
String	Nama menu, jenis menu, nama file	Menyimpan teks seperti nama makanan, jenis, dan nama file.

Integer (int)	Harga dan stok	Menyimpan nilai numerik harga dan jumlah stok.
Tuple	for i, (key, data) in enumerate(stock_dict.items()).	Hasil dari dict.items() berupa pasangan (key, value) yang merupakan tuple.
List (list)	["nama", "jenis", "harga", "stok"]	Digunakan untuk mengecek pilihan pengurutan yang valid.

Alasan Pemilihan Struktur Data:

Program ini menggunakan beberapa struktur data, yaitu dictionary, nested dictionary, list, tuple, string, dan integer, yang dipilih sesuai dengan kebutuhan pengelolaan data menu cafe. Struktur data utama yang digunakan adalah dictionary karena memungkinkan proses pencarian, penambahan, pengubahan, dan penghapusan data berdasarkan nama menu sebagai key secara cepat dan efisien. Setiap menu disimpan dalam nested dictionary yang berisi atribut Nama, Jenis, Harga, dan Stok, sehingga informasi setiap menu tersusun dengan rapi dan mudah diakses. Sementara itu, string digunakan untuk menyimpan data berupa teks seperti nama menu, jenis menu, dan nama file, sedangkan integer digunakan untuk menyimpan nilai numerik seperti harga dan stok agar dapat dilakukan operasi perhitungan maupun perbandingan.

Selain itu, program juga memanfaatkan list dan tuple sebagai struktur data pendukung. List digunakan untuk menyimpan kumpulan pilihan yang valid, seperti kategori pengurutan (nama, jenis, harga, dan stok), sehingga memudahkan proses validasi input dari pengguna. Tuple digunakan saat melakukan iterasi menggunakan dictionary.items(), karena setiap elemen yang dihasilkan berupa pasangan key dan value yang bersifat tetap selama proses pembacaan data. Kombinasi seluruh struktur data tersebut membuat program lebih efisien, mudah dipelihara, dan mempermudah pengelolaan data menu cafe, terutama dalam melakukan operasi pencarian, penambahan, pengubahan, penghapusan, dan pengurutan data.

2.3 Algoritma yang Digunakan

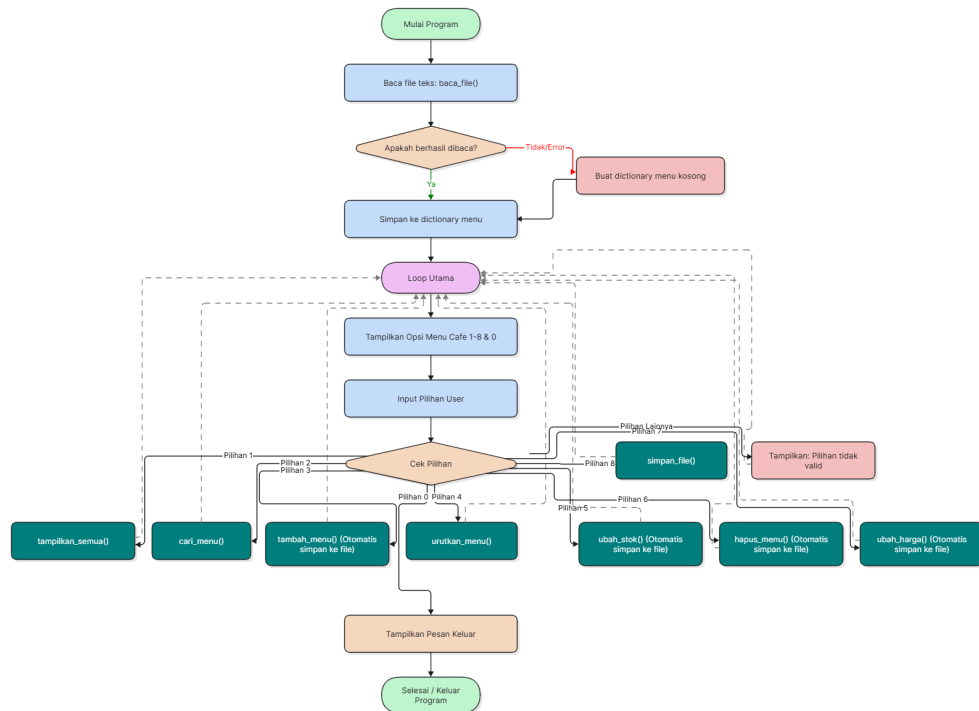
Searching:

- Algoritma: Direct Key Lookup pada Dictionary (Hash-based Search)
- Kompleksitas Waktu: $O(1)$ rata-rata
- Cara Kerja: Sistem mengonversi input nama menu dari pengguna menjadi huruf kecil (lowercase), kemudian langsung mengakses Dictionary menggunakan key tersebut. Jika key ditemukan, data menu ditampilkan; jika tidak, pesan "Menu tidak ditemukan!" ditampilkan kepada pengguna.
- Alasan Penggunaan: Dictionary hash-based lookup dipilih karena memberikan kecepatan pencarian yang konstan terlepas dari jumlah data yang tersimpan. Pendekatan ini jauh lebih optimal dibandingkan linear search tradisional, terutama untuk dataset besar seperti daftar menu café yang dapat berisi ratusan item.

Sorting:

- Algoritma: Python Built-in Timsort melalui fungsi `sorted()` dengan lambda comparator
- Kompleksitas Waktu: $O(n \log n)$ untuk kasus rata-rata dan terburuk
- Cara Kerja: Fungsi `urutkan_menu()` menerima input kriteria pengurutan dari pengguna (nama/jenis/harga/stok), kemudian memanggil `sorted()` pada `items()` dari Dictionary. Lambda comparator mengekstrak nilai atribut yang dipilih dari setiap item sebagai kunci pembandingan, menghasilkan list terurut yang ditampilkan ke terminal.
- Alasan Penggunaan: Python's Timsort merupakan algoritma hybrid yang menggabungkan Merge Sort dan Insertion Sort. Algoritma ini stabil (mempertahankan urutan relatif elemen yang sama) dan sangat efisien untuk data yang sudah sebagian terurut, yang merupakan kondisi umum pada daftar menu yang telah ada sebelumnya.

2.4 Diagram Alir Program (Flowchart)



2.5 Struktur Program

Program terdiri dari dua file utama:

- `Praktikum.py` – File utama yang memuat seluruh logika program, termasuk definisi fungsi CRUD, searching, sorting, file handling, dan fungsi `main()` sebagai entry point.
- `MenuCafe.txt` – File penyimpanan data menu café dalam format CSV (Comma-Separated Values) dengan kolom: `NamaMenu`, `JenisMenu`, `Harga`, `Stok`.

Fungsi-fungsi utama dalam `Praktikum.py`:

- `baca_file(nama_file)` – Membaca dan mem-parsing file TXT ke dalam Dictionary.
- `simpan_file(nama_file, stock_dict)` – Menulis seluruh data Dictionary ke file TXT.
- `tampilkan_semua(stock_dict)` – Menampilkan seluruh menu dalam format tabel.
- `cari_menu(stock_dict)` – Mencari menu berdasarkan nama (key lookup).
- `tambah_menu(stock_dict)` – Menambahkan item menu baru ke Dictionary dan menyimpannya.

- `ubah_harga(stock_dict)` – Memperbarui harga menu yang dipilih.
- `ubah_stok(stock_dict)` – Memperbarui stok menu yang dipilih.
- `hapus_menu(stock_dict)` – Menghapus item menu dari Dictionary dan menyinkronisasi ke file.
- `urutkan_menu(stock_dict)` – Mengurutkan menu berdasarkan kriteria yang dipilih pengguna.
- `main()` – Fungsi utama yang menjalankan loop program dan menangani interaksi pengguna.

BAB III. IMPLEMENTASI PROGRAM

3.1 Tampilan Menu Utama

Saat program dijalankan, pengguna akan disambut dengan tampilan menu utama berikut:

```
=== MENU CAFE ===  
1. Tampilkan semua menu  
2. Cari menu  
3. Tambah menu  
4. Urutkan menu  
5. Ubah stok menu  
6. Hapus menu  
7. Ubah harga menu  
8. Simpan ke file  
0. Keluar  
Silahkan pilih: 
```

Menu utama menyediakan 9 pilihan operasi yang mencakup seluruh fungsionalitas sistem. Desain menu yang sederhana dan intuitif memudahkan pengguna dalam mengoperasikan aplikasi.

3.2 Fitur Create (Tambah Menu)

```
Masukkan nama menu baru: Cakwe  
Masukkan jenis menu: Snack  
Masukkan harga menu: 20000  
Masukkan stok menu: 15  
Menu 'Cakwe' berhasil ditambahkan.
```

Fitur tambah menu diimplementasikan melalui fungsi `tambah_menu()`. Pengguna diminta untuk memasukkan nama menu baru, jenis menu (contoh: Makanan, Minuman, Snack), harga, dan stok. Sistem akan memvalidasi apakah nama menu yang dimasukkan sudah ada dalam Dictionary. Jika nama belum terdaftar, data baru akan ditambahkan ke

Dictionary dan secara otomatis disimpan ke file MenuCafe.txt melalui pemanggilan fungsi `simpan_file()`. Fitur ini memungkinkan pengelola café untuk menambahkan item menu baru kapan saja tanpa harus mengedit file secara langsung.

3.3 Fitur Read (Tampilkan)

Menu Cafe				
=====				
Nama Makanan		Jenis	Harga	Stok
=====				
1.	Nasi Goreng	Makanan	15000	20
2.	Mie Goreng	Makanan	14000	25
3.	Ayam Bakar	Makanan	20000	15
4.	Ayam Goreng	Makanan	18000	18
5.	Sate Ayam	Makanan	22000	12
6.	Sate Kambing	Makanan	30000	10
7.	Bakso	Makanan	13000	30
8.	Mie Ayam	Makanan	12000	28
9.	Soto Ayam	Makanan	17000	16
10.	Soto Betawi	Makanan	25000	8

Fitur Read terbagi menjadi dua subfungsi. Pertama, fungsi `tampilkan_semua()` menampilkan seluruh data menu dalam format tabel yang rapi dengan kolom Nama Makanan, Jenis, Harga, dan Stok. Tabel menggunakan pemformatan string Python (f-string dengan spesifikasi lebar kolom) untuk menghasilkan tampilan yang konsisten. Kedua, fungsi `cari_menu()` memungkinkan pencarian spesifik berdasarkan nama menu menggunakan mekanisme key lookup Dictionary. Hasil pencarian menampilkan detail lengkap menu yang ditemukan, atau pesan kesalahan jika menu tidak ada.

3.4 Fitur Update (Ubah Harga dan Stok)

```
Masukkan nama menu yang ingin diubah harganya: Bakwan
Masukkan harga baru: 16000
Harga menu 'Bakwan' berhasil diubah menjadi 16000.
```

```
Masukkan nama menu yang ingin diubah stoknya: Cakwe
Masukkan stok baru: 20
Stok menu 'Cakwe' berhasil diubah menjadi 20.
```

Fitur Update diimplementasikan dalam dua fungsi terpisah: `ubah_harga()` dan `ubah_stok()`. Pada `ubah_harga()`, pengguna memasukkan nama menu yang ingin diperbarui harganya. Sistem melakukan validasi keberadaan menu dalam Dictionary, lalu meminta input harga baru. Setelah validasi tipe data (memastikan input berupa angka), nilai harga pada Dictionary diperbarui dan file disimpan ulang. Mekanisme yang sama berlaku pada `ubah_stok()`. Pemisahan dua fungsi update ini mengikuti prinsip Single Responsibility, memudahkan pemeliharaan dan pengembangan kode.

3.5 Fitur Delete (Hapus Menu)

```
Silahkan pilih: 6
Masukkan nama menu yang ingin dihapus: Cakwe
Menu 'Cakwe' berhasil dihapus.
```

Fitur hapus menu diimplementasikan melalui fungsi `hapus_menu()`. Pengguna memasukkan nama menu yang ingin dihapus. Sistem mencari key yang sesuai dalam Dictionary (setelah konversi ke lowercase), kemudian menghapus pasangan key-value tersebut menggunakan operator del Python. Setelah penghapusan berhasil, fungsi `simpan_file()` dipanggil untuk menyinkronisasi perubahan ke file penyimpanan, memastikan data yang terhapus tidak muncul kembali pada sesi program berikutnya.

3.6 Fitur Searching

```
Silahkan pilih: 2

Masukkan nama menu: Bakwan
Menu ditemukan!
Menu : Bakwan | Jenis : Snack | Harga : 5000 | Stok : 45
```

Algoritma pencarian yang digunakan adalah Direct Key Lookup pada struktur data Dictionary. Implementasinya ada pada fungsi `cari_menu()` yang menerima input nama menu dari pengguna, mengonversinya ke lowercase, kemudian langsung mengakses Dictionary dengan ekspresi `if nama in stock_dict`. Pendekatan ini memanfaatkan sifat hash table dari Dictionary Python yang memiliki kompleksitas rata-rata $O(1)$. Berbeda dengan

Sequential Search yang harus mengiterasi seluruh elemen satu per satu ($O(n)$), key lookup pada Dictionary hampir tidak terpengaruh oleh penambahan jumlah data.

3.7 Fitur Sorting

```
Urutkan berdasarkan (nama/jenis/harga/stok): nama
1. Air Mineral | Jenis : Minuman | Harga : 4000 | Stok : 100
2. Ayam Bakar | Jenis : Makanan | Harga : 20000 | Stok : 15
3. Ayam Geprek | Jenis : Makanan | Harga : 18000 | Stok : 20
4. Ayam Goreng | Jenis : Makanan | Harga : 18000 | Stok : 18
5. Bakso | Jenis : Makanan | Harga : 13000 | Stok : 30
6. Bakwan | Jenis : Snack | Harga : 5000 | Stok : 45
7. Batagor | Jenis : Snack | Harga : 12000 | Stok : 21
8. Brownies | Jenis : Snack | Harga : 20000 | Stok : 9
9. Bubur Ayam | Jenis : Makanan | Harga : 12000 | Stok : 23
10. Bubur Kacang Hijau | Jenis : Snack | Harga : 10000 | Stok : 20
```

Algoritma pengurutan yang digunakan adalah Timsort—algoritma bawaan Python—yang diakses melalui fungsi `sorted()`. Implementasinya ada pada fungsi `urutkan_menu()` yang mendukung pengurutan berdasarkan empat kriteria: nama, jenis, harga, dan stok. Fungsi ini memanggil `sorted(stock_dict.items(), key=lambda x: x[1][urutan.capitalize()])`, di mana lambda berfungsi sebagai comparator yang mengekstrak nilai atribut yang dipilih dari setiap item Dictionary. Timsort dipilih karena stabilitasnya dan efisiensinya pada data yang sebagian besar sudah terurut. Hasil pengurutan ditampilkan langsung ke terminal dalam format tabel.

3.8 Penyimpanan Data (File Handling)

```
8. Simpan ke file
0. Keluar
Silahkan pilih: 8
Data berhasil disimpan.
```

```
Nasi Goreng,Makanan,15000,20
Mie Goreng,Makanan,14000,25
Ayam Bakar,Makanan,20000,15
Ayam Goreng,Makanan,18000,18
Sate Ayam,Makanan,22000,12
Sate Kambing,Makanan,30000,10
Bakso,Makanan,13000,30
Mie Ayam,Makanan,12000,28
Soto Ayam,Makanan,17000,16
Soto Betawi,Makanan,25000,8
Rendang,Makanan,35000,7
Gulai Ayam,Makanan,23000,9
Rawon,Makanan,27000,6
Nasi Uduk,Makanan,10000,22
Nasi Kuning,Makanan,12000,19
Lontong Sayur,Makanan,11000,17
Nasi Campur,Makanan,20000,12
Nasi Padang,Makanan,25000,10
Ayam Geprek,Makanan,18000,20
```

Format file yang digunakan adalah TXT berformat CSV (Comma-Separated Values) dengan nama MenuCafe.txt. Setiap baris merepresentasikan satu item menu dengan empat field yang dipisahkan koma: NamaMenu, JenisMenu, Harga, Stok.

Cara membaca file: Fungsi `baca_file()` membuka file dengan encoding UTF-8, membaca setiap baris, menghapus whitespace di awal dan akhir (`strip()`), kemudian memisahkan empat field menggunakan metode `split(",")`. Data Harga dan Stok dikonversi ke tipe Integer. Hasil parsing disimpan sebagai nested Dictionary dengan key nama menu lowercase.

```
file = "MenuCafe.txt"
stock_dict = {}

def baca_file(nama_file):
    data = {}
    try:
        with open(nama_file, "r", encoding="utf-8") as f:
            for baris in f:
                baris = baris.strip()
                if not baris:
                    continue
                try:
                    nama, jenis, harga, stok = baris.split(",")
                    data[nama.lower()] = {
                        "Nama": nama,
                        "Jenis": jenis,
                        "Harga": int(harga),
                        "Stok": int(stok)
                    }
                except ValueError:
                    print(f"Format salah pada baris: {baris}")
            except FileNotFoundError:
                print("File tidak ditemukan, akan dibuat baru.")
    return data
```

Cara menyimpan file: Fungsi `simpan_file()` membuka file dalam mode tulis ("w") dengan encoding UTF-8, kemudian mengiterasi seluruh item Dictionary dan menulis setiap item dalam format "Nama,Jenis,Harga,Stok\n". Fungsi ini dipanggil secara otomatis setiap kali terjadi perubahan data (tambah, ubah, hapus).

```
def simpan_file(nama_file, stock_dict):
    with open(nama_file, "w", encoding="utf-8") as f:
        for key in stock_dict:
            data = stock_dict[key]
            nama = data["Nama"]
            jenis = data["Jenis"]
            harga = data["Harga"]
            stok = data["Stok"]
            f.write(f"{nama},{jenis},{harga},{stok}\n")
```

3.9 Validasi Input dan Error Handling

Kesalahan Input	Penanganan oleh Sistem
Huruf/teks pada input angka (harga/stok)	Blok try-except ValueError menangkap exception dan menampilkan pesan "Harga dan stok harus angka." Program tidak crash dan kembali ke menu utama.
Nama menu tidak ditemukan pada operasi ubah/hapus/cari	Sistem menampilkan pesan "Menu tidak ditemukan." dan menghentikan operasi tanpa modifikasi data.
File MenuCafe.txt belum tersedia saat program pertama dijalankan	Blok try-except FileNotFoundError pada fungsi <code>baca_file()</code> menangkap exception, mencetak pesan informatif, dan mengembalikan Dictionary kosong. File akan dibuat otomatis saat pengguna pertama kali menambahkan data baru.
Nama menu duplikat pada operasi tambah	Sistem memeriksa keberadaan key dalam Dictionary sebelum menambahkan. Jika sudah ada, pesan "Nama

	menu sudah digunakan." ditampilkan dan operasi dibatalkan.
Pilihan menu tidak valid di menu utama	Blok else pada struktur if-elif-else di fungsi main() menampilkan "Pilihan tidak valid." dan program kembali menampilkan menu utama.
Kriteria sorting tidak valid	Sistem memvalidasi input kriteria sorting terhadap list ["nama", "jenis", "harga", "stok"]. Jika tidak valid, pesan "Pilihan urutan tidak valid." ditampilkan.

BAB IV. PENGUJIAN DAN ANALISIS

4.1 Skenario Pengujian

No	Skenario	Langkah Uji	Hasil
1	Tambah menu baru	Pilih opsi 3, masukkan nama "Kopi Arabika", jenis "Minuman", harga 22000, stok 15	Berhasil. Data tersimpan di Dictionary dan file.
2	Tampilkan semua menu	Pilih opsi 1, lihat daftar menu dalam format tabel	Berhasil. Semua 98 item menu ditampilkan dengan format kolom yang rapi.
3	Cari menu yang ada	Pilih opsi 2, masukkan nama "Nasi Goreng"	Berhasil. Detail menu ditampilkan: Jenis Makanan, Harga 15000, Stok 20.
4	Cari menu tidak ada	Pilih opsi 2, masukkan nama "Nasi Padang Spesial"	Berhasil. Pesan "Menu tidak ditemukan!" ditampilkan.
5	Ubah harga menu	Pilih opsi 7, masukkan nama "Es Teh", harga baru 6000	Berhasil. Harga diperbarui di Dictionary dan file tersinkronisasi.
6	Ubah stok menu	Pilih opsi 5, masukkan nama "Bakso", stok baru 50	Berhasil. Stok diperbarui dan file disimpan otomatis.
7	Hapus menu	Pilih opsi 6, masukkan nama "Telur Dadar"	Berhasil. Item dihapus dari

			Dictionary dan tidak muncul di file.
8	Sorting berdasarkan harga	Pilih opsi 4, masukkan kriteria "harga"	Berhasil. Menu ditampilkan terurut dari harga terendah (Kerupuk Rp3.000) ke tertinggi (Steak Sapi Rp45.000).
9	Sorting berdasarkan jenis	Pilih opsi 4, masukkan kriteria "jenis"	Berhasil. Menu dikelompokkan berdasarkan jenis (Makanan, Minuman, Snack).
10	Input harga berupa teks	Pilih opsi 7, masukkan nama menu valid, masukkan "dua puluh ribu" sebagai harga	Berhasil. Pesan "Harga harus angka." ditampilkan, program tidak crash.
11	Tambah menu duplikat	Pilih opsi 3, masukkan nama "Nasi Goreng" yang sudah ada	Berhasil. Pesan "Nama menu sudah digunakan." ditampilkan.
12	Persistensi data	Tambah menu baru, tutup program, jalankan ulang, tampilkan semua menu	Berhasil. Data yang ditambahkan sebelumnya masih ada dan terbaca dengan benar.

4.2 Kelebihan Sistem

1. Data tersimpan secara permanen menggunakan file TXT format CSV, sehingga tidak hilang ketika program ditutup dan dapat diakses kembali pada sesi berikutnya.

2. Program memiliki ketahanan terhadap kesalahan input (fault-tolerant) berkat mekanisme validasi dan exception handling yang komprehensif, sehingga tidak mudah crash akibat input pengguna yang tidak valid.
3. Pencarian menu menggunakan Dictionary key lookup dengan kompleksitas $O(1)$ memberikan kecepatan pencarian yang konstan dan tidak terpengaruh oleh penambahan jumlah data.
4. Antarmuka menu yang sederhana dan deskriptif memudahkan pengguna dalam mengoperasikan semua fitur tanpa memerlukan panduan tambahan.
5. Fitur sorting multi-kriteria (nama, jenis, harga, stok) memberikan fleksibilitas kepada pengelola dalam memandang dan menganalisis data menu.
6. Arsitektur fungsi yang modular (setiap operasi memiliki fungsi tersendiri) memudahkan pemeliharaan, pengujian, dan pengembangan kode lebih lanjut.

4.3 Keterbatasan Sistem

1. Antarmuka masih berbasis CLI (Command Line Interface) yang kurang intuitif dibandingkan aplikasi berbasis GUI (Graphical User Interface) atau web, sehingga belum ramah untuk pengguna awam yang tidak terbiasa dengan terminal.
2. Penyimpanan data masih menggunakan file teks flat (TXT/CSV), belum menggunakan sistem manajemen basis data relasional (RDBMS) seperti SQLite atau MySQL yang lebih terstruktur dan mendukung query kompleks.
3. Sistem belum mendukung multiuser secara bersamaan; tidak ada mekanisme penguncian file (file locking) sehingga data dapat rusak jika dua instansi program dijalankan secara bersamaan dan keduanya menulis ke file yang sama.
4. Fitur pencarian hanya mendukung pencocokan tepat (exact match) berdasarkan nama menu. Belum ada dukungan untuk pencarian parsial (substring search) atau pencarian berdasarkan kriteria lain seperti jenis atau rentang harga.
5. Tidak ada fitur autentikasi atau kontrol akses pengguna, sehingga semua pengguna yang dapat mengakses sistem memiliki hak yang sama untuk memodifikasi data.

BAB V. KENDALA DAN SOLUSI

Kendala	Solusi
Data menu hilang ketika program ditutup pada iterasi pertama pengembangan	Mengimplementasikan mekanisme file handling menggunakan file TXT format CSV. Fungsi <code>baca_file()</code> membaca data saat startup dan <code>simpan_file()</code> dipanggil otomatis setiap kali terjadi perubahan data.
Program crash saat pengguna memasukkan teks pada kolom harga atau stok yang seharusnya angka	Mengimplementasikan blok <code>try-except ValueError</code> pada fungsi <code>tambah_menu()</code> , <code>ubah_harga()</code> , dan <code>ubah_stok()</code> . Exception ditangkap dan pesan kesalahan yang informatif ditampilkan tanpa menghentikan program.
Pencarian menu tidak menemukan hasil karena perbedaan kapitalisasi (case sensitivity)	Menstandarkan key Dictionary menjadi lowercase menggunakan metode <code>.lower()</code> pada saat penambahan data maupun pencarian. Hal ini memastikan pencarian case-insensitive yang konsisten.
Tampilan tabel menu tidak rapi karena perbedaan panjang nama menu	Menggunakan f-string dengan spesifikasi lebar kolom tetap (misalnya <code>{data['Nama']:<25}</code>) untuk memastikan setiap kolom memiliki lebar yang konsisten dalam tampilan tabel terminal.
Beberapa baris pada <code>MenuCafe.txt</code> memiliki format yang tidak konsisten menyebabkan error parsing	Menambahkan blok <code>try-except ValueError</code> dalam fungsi <code>baca_file()</code> pada level iterasi baris. Baris yang formatnya salah akan dilewati dengan menampilkan pesan peringatan tanpa menghentikan keseluruhan proses pembacaan file.

Pengalaman yang paling menantang selama pengerjaan proyek ini adalah memastikan konsistensi antara data yang ada di memori (Dictionary) dan data yang tersimpan di file. Awalnya, terdapat kasus di mana data berhasil diubah di Dictionary

namun tidak tersimpan ke file karena lupa memanggil `simpan_file()`. Hal ini menyebabkan perubahan hilang ketika program dijalankan ulang. Solusinya adalah dengan memastikan setiap fungsi yang memodifikasi data (`tambah_menu`, `ubah_harga`, `ubah_stok`, `hapus_menu`) selalu diakhiri dengan pemanggilan `simpan_file()` secara eksplisit.

Tantangan lain adalah memahami dan menerapkan lambda function sebagai comparator pada fungsi `sorted()` untuk mendukung sorting multi-kriteria. Diperlukan pemahaman mendalam tentang cara Python mengiterasi Dictionary items() dan bagaimana `.capitalize()` diperlukan untuk mencocokkan key Dictionary yang menggunakan huruf kapital pada huruf pertama ("Nama", "Jenis", "Harga", "Stok").

BAB VI. PEMBAGIAN TUGAS ANGGOTA

Nama / NIM	Tugas dan Kontribusi
Muhammad Yusuf Baihaqi Bawono (J0403251096)	Perancangan arsitektur sistem, implementasi fungsi <code>baca_file()</code> dan <code>simpan_file()</code> , serta fitur Create (<code>tambah_menu</code>) dan Delete (<code>hapus_menu</code>). Bertanggung jawab atas mekanisme file handling dan persistensi data.
Yasmin Khoirun Nisa' (J0403251071)	Implementasi fitur Read (<code>tampilkan_semua</code> , <code>cari_menu</code>) dan Update (<code>ubah_harga</code> , <code>ubah_stok</code>). Bertanggung jawab atas format tampilan tabel dan mekanisme validasi input.
Asma Mazaya (J0403251105)	Implementasi fitur Sorting (<code>urutkan_menu</code>) dan integrasi fungsi <code>main()</code> sebagai entry point. Bertanggung jawab atas pengujian sistem, penulisan laporan, dan pengelolaan repositori GitHub.

Seluruh anggota kelompok berkontribusi dalam penyusunan dataset `MenuCafe.txt` yang mencakup 98 item menu dari berbagai kategori (Makanan, Minuman, Snack). Kontribusi setiap anggota dapat dilihat secara transparan melalui riwayat commit pada repositori GitHub: https://github.com/Dokyaroky/ASD_Kelompok16

BAB VII. KESIMPULAN DAN SARAN

Kesimpulan

1. Sistem Manajemen Menu dan Harga Cafe (MenuCafe Management System) berhasil dikembangkan sebagai aplikasi CLI berbasis Python yang mampu melakukan operasi CRUD lengkap terhadap data menu café, dilengkapi fitur searching berbasis Dictionary key lookup dan sorting multi-kriteria menggunakan Timsort melalui fungsi `sorted()` Python.
2. Implementasi Dictionary sebagai struktur data utama terbukti efektif dalam mengelola dataset berisi 98 item menu, memberikan kecepatan pencarian $O(1)$ yang konstan dan operasi penambahan/penghapusan yang efisien tanpa perlu melakukan pergeseran elemen seperti pada struktur data List.
3. Mekanisme file handling berbasis file TXT format CSV berhasil menjamin persistensi data antar sesi program, dengan validasi input dan exception handling yang komprehensif menjadikan sistem robust terhadap kesalahan penggunaan.

Saran Pengembangan

1. Mengembangkan antarmuka berbasis GUI menggunakan library seperti Tkinter atau PyQt, atau versi web menggunakan Flask/Django, agar lebih mudah digunakan oleh pengelola café yang tidak terbiasa dengan terminal.
2. Mengganti penyimpanan file teks dengan sistem manajemen basis data relasional seperti SQLite yang terintegrasi dengan Python, untuk mendukung query yang lebih kompleks, integritas data yang lebih baik, dan kemampuan multiuser.
3. Menambahkan fitur pencarian parsial (substring search) atau fuzzy search sehingga pengguna tidak perlu mengingat nama menu secara persis untuk melakukan pencarian.
4. Menambahkan fitur manajemen kategori menu yang dinamis, laporan penjualan, dan integrasi dengan sistem Point of Sale (POS) untuk menjadikan sistem lebih komprehensif sebagai alat manajemen bisnis café.

5. Mengimplementasikan mekanisme autentikasi pengguna dengan berbagai tingkat hak akses (admin, kasir) untuk meningkatkan keamanan dan kontrol terhadap data menu.

DAFTAR PUSTAKA

Cormen, T.H., Leiserson, C.E., Rivest, R.L. & Stein, C., 2022. *Introduction to Algorithms*. 4th ed. Cambridge, MA: MIT Press.

Goodrich, M.T., Tamassia, R. & Goldwasser, M.H., 2013. *Data Structures and Algorithms in Python*. Hoboken, NJ: John Wiley & Sons.

Miller, B. & Ranum, D., 2011. *Problem Solving with Algorithms and Data Structures Using Python*. Franklin: Franklin, Beedle & Associates.

Python Software Foundation, 2024. *Python 3 Documentation: Built-in Types – Dictionaries*. Available at: <https://docs.python.org/3/library/stdtypes.html#dict>

Python Software Foundation, 2024. *Sorting HOW TO*. Available at: <https://docs.python.org/3/howto/sorting.html>

Sedgewick, R. & Wayne, K., 2011. *Algorithms*. 4th ed. Upper Saddle River, NJ: Addison-Wesley.

LAMPIRAN

Lampiran A. Link GitHub

Repository Proyek: https://github.com/Dokyaroky/ASD_Kelompok16

Lampiran B. Screenshot Program

```
=== MENU CAFE ===
1. Tampilkan semua menu
2. Cari menu
3. Tambah menu
4. Urutkan menu
5. Ubah stok menu
6. Hapus menu
7. Ubah harga menu
8. Simpan ke file
0. Keluar
Silahkan pilih: █
```

Menu Cafe

=====			
Nama Makanan	Jenis	Harga	Stok
=====			
1. Nasi Goreng	Makanan	15000	20
2. Mie Goreng	Makanan	14000	25
3. Ayam Bakar	Makanan	20000	15
4. Ayam Goreng	Makanan	18000	18
5. Sate Ayam	Makanan	22000	12
6. Sate Kambing	Makanan	30000	10
7. Bakso	Makanan	13000	30
8. Mie Ayam	Makanan	12000	28
9. Soto Ayam	Makanan	17000	16
10. Soto Betawi	Makanan	25000	8

04. Keluar

Silahkan pilih: 2

Masukkan nama menu: Bakwan

Menu ditemukan!

Menu : Bakwan | Jenis : Snack | Harga : 5000 | Stok : 45

Masukkan nama menu baru: Cakwe

Masukkan jenis menu: Snack

Masukkan harga menu: 20000

Masukkan stok menu: 15

Menu 'Cakwe' berhasil ditambahkan.

99.	Cakwe	Snack	20000	15
-----	-------	-------	-------	----

Urutkan berdasarkan (nama/jenis/harga/stok): nama

1. Air Mineral | Jenis : Minuman | Harga : 4000 | Stok : 100
2. Ayam Bakar | Jenis : Makanan | Harga : 20000 | Stok : 15
3. Ayam Geprek | Jenis : Makanan | Harga : 18000 | Stok : 20
4. Ayam Goreng | Jenis : Makanan | Harga : 18000 | Stok : 18
5. Bakso | Jenis : Makanan | Harga : 13000 | Stok : 30
6. Bakwan | Jenis : Snack | Harga : 5000 | Stok : 45
7. Batagor | Jenis : Snack | Harga : 12000 | Stok : 21
8. Brownies | Jenis : Snack | Harga : 20000 | Stok : 9
9. Bubur Ayam | Jenis : Makanan | Harga : 12000 | Stok : 23
10. Bubur Kacang Hijau | Jenis : Snack | Harga : 10000 | Stok : 20
11. Buntut | Jenis : Makanan | Harga : 80000 | Stok : 15

Masukkan nama menu yang ingin diubah stoknya: Cakwe

Masukkan stok baru: 20

Stok menu 'Cakwe' berhasil diubah menjadi 20.

Silahkan pilih: 6

Masukkan nama menu yang ingin dihapus: Cakwe

Menu 'Cakwe' berhasil dihapus.

```
Masukkan nama menu yang ingin diubah harganya: Bakwan
Masukkan harga baru: 16000
Harga menu 'Bakwan' berhasil diubah menjadi 16000.
```

```
8. Simpan ke file
0. Keluar
Silahkan pilih: 8
Data berhasil disimpan.
```

```
=== MENU CAFE ===
1. Tampilkan semua menu
2. Cari menu
3. Tambah menu
4. Urutkan menu
5. Ubah stok menu
6. Hapus menu
7. Ubah harga menu
8. Simpan ke file
0. Keluar
Silahkan pilih: 0
Terima kasih telah menggunakan program ini!
```

Lampiran C. Struktur Folder Proyek

Program Manajemen Menu Cafe

```
|
|
|— Variabel Global
|   |— file
|   |— stock_dict
|
|
|— Fungsi Input
|   |— baca_file()
```

```

|
├─ Fungsi Output
|   └─ simpan_file()
|
|
├─ Fungsi CRUD
|   ├─ tampilkan_semua()
|   ├─ cari_menu()
|   ├─ tambah_menu()
|   ├─ ubah_harga()
|   ├─ ubah_stok()
|   └─ hapus_menu()
|
|
├─ Fungsi Pengolahan Data
|   └─ urutkan_menu()
|
|
├─ Fungsi Pendukung
|   ├─ simpan_data()
|   └─ clear_screen()
|
|
├─ Fungsi Main
|   └─ main()
|
|
└─ Entry Point
    └─ if __name__ == "__main__":

```

Lampiran D. Commit History GitHub

Commits on Jun 26, 2026
Add YouTube video link for project presentation yasminkhoirunnisa2007-source authored 16 minutes ago
Commits on Jun 25, 2026
Video Presentasi Kelompok 16 yasminkhoirunnisa2007-source committed 16 hours ago
Commits on Mar 6, 2026
Asma paradoxdelulu committed on Mar 6
yasmin yasminkhoirunnisa2007-source committed on Mar 6
Update Praktikum.py Dokyaroky committed on Mar 6
Test1 Dokyaroky committed on Mar 6
Commits on Feb 27, 2026
Create Praktikum.py Dokyaroky committed on Feb 27
Commits on Feb 20, 2026
Txt Dokyaroky committed on Feb 20
test paradoxdelulu committed on Feb 20
Initial commit Dokyaroky committed on Feb 20

Lampiran E. Source Code Utama

```
file = "MenuCafe.txt"

stock_dict = {}

def baca_file(nama_file):

    data = {}

    try:

        with open(nama_file, "r", encoding="utf-8") as f:

            for baris in f:

                baris = baris.strip()

                if not baris:

                    continue

                try:

                    nama, jenis, harga, stok = baris.split(",")

                    data[nama.lower()] = {

                        "Nama": nama,

                        "Jenis": jenis,

                        "Harga": int(harga),

                        "Stok": int(stok)

                    }

                except ValueError:

                    print(f"Format salah pada baris: {baris}")

    except FileNotFoundError:

        print("File tidak ditemukan, akan dibuat baru.")

    return data
```

```

def simpan_file(nama_file, stock_dict):

    with open(nama_file, "w", encoding="utf-8") as f:

        for key in stock_dict:

            data = stock_dict[key]

            nama = data["Nama"]

            jenis = data["Jenis"]

            harga = data["Harga"]

            stok = data["Stok"]

            f.write(f"{nama},{jenis},{harga},{stok}\n")

def tampilkan_semua(stock_dict):

    if not stock_dict:

        print("Menu sedang kosong")

        return

    print("\nMenu Cafe")

    print("=" * 62)

    print((f"{'Nama Makanan' : <31} | {'Jenis' : <10} | {'Harga' : <8} | {'Stok' : <10}"))

    print("=" * 62)

    for i, (key, data) in enumerate(stock_dict.items(), start=1):

        print(f"{str(i)+'.' : <5} {data['Nama'] : <25} | {data['Jenis'] : <10} | {data['Harga'] : <8} | {data['Stok'] : <10}")

def cari_menu(stock_dict):

    nama = input("\nMasukkan nama menu: ").strip().lower()

    if nama in stock_dict:

        data = stock_dict[nama]

```

```

        print("Menu ditemukan!")

        print(f"Menu : {data['Nama']} | Jenis : {data['Jenis']} |
Harga : {data['Harga']} | Stok : {data['Stok']}")

    else:

        print("Menu tidak ditemukan!")

def tambah_menu(stock_dict):

    nama = input("Masukkan nama menu baru: ").strip()

    key = nama.lower()

    if key in stock_dict:

        print("Nama menu sudah digunakan.")

        return

    jenis = input("Masukkan jenis menu: ").strip()

    try:

        harga = int(input("Masukkan harga menu: "))

        stok = int(input("Masukkan stok menu: "))

    except ValueError:

        print("Harga dan stok harus angka.")

        return

    stock_dict[key] = {

        "Nama": nama,

        "Jenis": jenis,

        "Harga": harga,

        "Stok": stok

    }

    simpan_file(file, stock_dict)

    print(f"Menu '{nama}' berhasil ditambahkan.")

```



```

def ubah_harga(stock_dict):

    nama = input("Masukkan nama menu yang ingin diubah harganya: ")
    nama = nama.strip().lower()

    if nama not in stock_dict:

        print("Menu tidak ditemukan.")

        return

    try:

        harga_baru = int(input("Masukkan harga baru: "))

    except ValueError:

        print("Harga harus angka.")

        return

    nama_menu = stock_dict[nama]["Nama"]

    stock_dict[nama]["Harga"] = harga_baru

    simpan_file(file, stock_dict)

    print(f"Harga menu '{nama_menu}' berhasil diubah menjadi {harga_baru}.")


def ubah_stok(stock_dict):

    nama = input("Masukkan nama menu yang ingin diubah stoknya: ")
    nama = nama.strip().lower()

    if nama not in stock_dict:

        print("Menu tidak ditemukan.")

        return

    try:

        stok_baru = int(input("Masukkan stok baru: "))

```

```

except ValueError:

    print("Stok harus angka.")

    return

nama_menu = stock_dict[nama]["Nama"]

stock_dict[nama]["Stok"] = stok_baru

simpan_file(file, stock_dict)

    print(f"Stok menu '{nama_menu}' berhasil diubah menjadi
{stok_baru}.")

def hapus_menu(stock_dict):

    nama = input("Masukkan nama menu yang ingin dihapus:
").strip().lower()

    if nama in stock_dict:

        nama_menu = stock_dict[nama]["Nama"]

        del stock_dict[nama]

        simpan_file(file, stock_dict)

        print(f"Menu '{nama_menu}' berhasil dihapus.")

    else:

        print("Menu tidak ditemukan.")

def urutkan_menu(stock_dict):

    urutan = input("Urutkan berdasarkan (nama/jenis/harga/stok):
").strip().lower()

    if urutan not in ["nama", "jenis", "harga", "stok"]:

        print("Pilihan urutan tidak valid.")

    return

```

```

        sorted_menu = sorted(stock_dict.items(), key=lambda x:
x[1][urutan.capitalize()])

        for i, (nama, data) in enumerate(sorted_menu, start=1):

            print(f"{i}. {data['Nama']} | Jenis : {data['Jenis']} | Harga
: {data['Harga']} | Stok : {data['Stok']}")

def simpan_data():

    simpan_file(file, stock_dict)

def clear_screen():

    import os

    os.system('cls' if os.name == 'nt' else 'clear')

def main():

    menu = baca_file(file)

    while True:

        print("\n=== MENU CAFE ===")

        print("1. Tampilkan semua menu")

        print("2. Cari menu")

        print("3. Tambah menu")

        print("4. Urutkan menu")

        print("5. Ubah stok menu")

        print("6. Hapus menu")

        print("7. Ubah harga menu")

```

```

print("8. Simpan ke file")

print("0. Keluar")

pilihan = input("Silahkan pilih: ").strip()

if pilihan == "1":
    tampilkan_semua(menu)
elif pilihan == "2":
    cari_menu(menu)
elif pilihan == "3":
    tambah_menu(menu)
elif pilihan == "4":
    urutkan_menu(menu)
elif pilihan == "5":
    ubah_stok(menu)
elif pilihan == "6":
    hapus_menu(menu)
elif pilihan == "7":
    ubah_harga(menu)
elif pilihan == "8":
    simpan_file(file, menu)
    print("Data berhasil disimpan.")
elif pilihan == "0":
    print("Terima kasih telah menggunakan program ini!")
    break
else:
    print("Pilihan tidak valid.")

```

```
if __name__ == "__main__":  
    main()
```